INFORME PRÁCTICA FINAL ARQUITECTURA

PRESENTADO POR:
EDDY EMMANUEL SANCHEZ OBANDO
JOSEPH DAVID TRUJILLO GOMEZ
JUAN DAVID MEZA PAZ

PRESENTADO A:
FERNANDO APARICIO URBANO MOLANO

Universidad del Cauca

UNIVERSIDAD DEL CAUCA
FACULTAD DE INGENIERÍA ELECTRÓNICA Y TELECOMUNICACIONES
ARQUITECTURA COMPUTACIONAL
POPAYÁN - CAUCA
ABRIL 2026

1. Abstract 3
2. Introducción 4
2.1 Problema de estudio 4
2.2 Objetivos 4
2.3 Caso de estudio 4
3. Metodología 5
3.1 Código C 5
3.1.1 Código Original 5
3.1.2 Código Optimizado 5
3.1.3 Código Reducido (El que se usó en RARS y RIPES) 6
3.2 Código RISC-V 7
4. Resultados Experimentales 9
4.1 Hazards 9
Hazard 1 9
Hazard 2 10
4.2 CPI 11
4.3 Análisis Caché Original 12
4.3 Análisis Caché Optimizado 12
4.5 Cálculo AMAT 13
Original 13
Optimizado 13
Mejora porcentual 13
5. Análisis y Discusión 13
5.1 Análisis de Hazards 13
5.2 Impacto de Caché 14
6. Conclusiones 14
6.1 Hallazgos Principales 14
6.2 Aprendizajes de Arquitectura 15

1. Abstract

Este proyecto analiza el rendimiento arquitectónico de un algoritmo de rotación horizontal de imágenes implementado en C y ensamblador RISC-V. El problema principal es la alta latencia por fallos de caché y los riesgos de datos en un procesador segmentado de 5 etapas. Se implementaron dos versiones: una naive y una optimizada mediante bloqueo de 64 píxeles. Las pruebas se realizaron en RARS, RIPES y Cachegrind. Los resultados mostraron que los D1 misses se mantuvieron similares, pero la tasa de misses mejoró de 0.73% a 0.63%, disminuyendo el AMAT de 1.7 ns a 1.6 ns (mejora del 5.9%). Se identificaron dos hazards críticos. El CPI real fue de 1.86 ciclos por instrucción.

2. Introducción

2.1 Problema de estudio

La rotación horizontal de una imagen consiste en reflejar cada fila de píxeles horizontalmente. Desde el punto de vista arquitectónico, este algoritmo presenta un desafío porque el acceso a memoria no es secuencial: mientras se lee un píxel en [x,y], se escribe en [W-1-x,y]. Esta falta de localidad espacial genera una alta tasa de fallos en la caché L1.

2.2 Objetivos

- Implementar el algoritmo de rotación horizontal en C y ensamblador RISC-V.
- Identificar y analizar data hazards y control hazards en pipeline de 5 etapas.
- Medir los D1 misses usando Cachegrind.
- Optimizar el algoritmo mediante bloqueo para reducir fallos de caché.
- Calcular el CPI y el AMAT.

2.3 Caso de estudio

Se trabajó con una imagen en escala de grises de 10,000 x 10,000 píxeles (100 millones de píxeles, 100 MB en memoria).

3. Metodología

3.1 Código C

3.1.1 Código Original

```
#include <stdio.h>
#include <stdlib.h>

#define FILAS 320
#define COLUMNAS 313
#define TAMANIO (FILAS * COLUMNAS)

int main() {

	int (*matriz)[COLUMNAS] = malloc(FILAS * sizeof(*matriz));
	if (!matriz) return 1;

	for (int i = 0; i < FILAS; i++) {
		for (int j = 0; j < COLUMNAS; j++) {
			matriz[i][j] = i * COLUMNAS + j;
		}
	}

	for (int i = 0; i < FILAS; i++) {
		int izquierda = 0;
		int derecha = COLUMNAS - 1;

		while (izquierda < derecha) {
			int temp = matriz[i][izquierda];
			matriz[i][izquierda] = matriz[i][derecha];
			matriz[i][derecha] = temp;
			izquierda++;
			derecha--;
		}
	}
	free(matriz);
	return 0;
}
```

3.1.2 Código Optimizado

```
#include <stdio.h>
#include <stdlib.h>

#define FILAS 320
#define COLUMNAS 313
#define GRUPO 4

int main() {
```

```
	int (*matriz)[COLUMNAS] = malloc(FILAS * sizeof(*matriz));
	if (!matriz) return 1;


	for (int i = 0; i < FILAS; i++) {
		for (int j = 0; j < COLUMNAS; j++) {
			matriz[i][j] = i * COLUMNAS + j;
		}
	}

	for (int i = 0; i < FILAS; i++) {
		int izquierda = 0;
		int derecha = COLUMNAS - 1;

		while (izquierda < derecha) {

			int contador = (derecha - izquierda + 1) < GRUPO ? (derecha - izquierda + 1) :
GRUPO;

			for (int k = 0; k < contador; k++) {
				int temp = matriz[i][izquierda + k];
				matriz[i][izquierda + k] = matriz[i][derecha - k];
				matriz[i][derecha - k] = temp;
			}

			izquierda += contador;
			derecha -= contador;
		}
	}
	free(matriz);
	return 0;
}
```

3.1.3 Código Reducido (El que se usó en RARS y RIPES)

```
#include <stdio.h>

#define FILAS 5
#define COLS 5

int main() {
	int imagen[5][5] = {
		{0,1,0,0,0},
		{0,1,1,0,0},
		{1,1,1,1,0},
		{0,1,1,0,0},
		{0,1,0,0,0}
	};

	for (int i = 0; i < FILAS; i++) {
```

```
            int izq = 0;
            int der = COLS - 1;

            while (izq < der) {

                int temp = imagen[i][izq];
                imagen[i][izq] = imagen[i][der];
                imagen[i][der] = temp;

                izq++;
                der--;
            }
        }

        return 0;
}
```

3.2 Código RISC-V

```
.data

imagen:

   .word 0,1,0,0,0

   .word 0,1,1,0,0

   .word 1,1,1,1,0

   .word 0,1,1,0,0

   .word 0,1,0,0,0


.text

.globl main


main:

   la   t0, imagen

   li   t1, 0

   li   s0, 5


fila_loop:
```

```
    bge  t1, s0, fin

    li   t3, 0
    li   t4, 4

    mul  t6, t1, s0
    slli t6, t6, 2
    add  t6, t0, t6

while_loop:
    bge  t3, t4, next_fila

    slli t5, t3, 2
    add  a0, t6, t5

    slli t5, t4, 2
    add  a1, t6, t5

    lw   t5, 0(a0)
    lw   t2, 0(a1)
    sw   t2, 0(a0)
    sw   t5, 0(a1)

    addi t3, t3, 1
    addi t4, t4, -1
    j    while_loop

next_fila:
    addi t1, t1, 1
    j    fila_loop
```

```
fin:
    li   a7, 10
    ecall
```

4. Resultados Experimentales

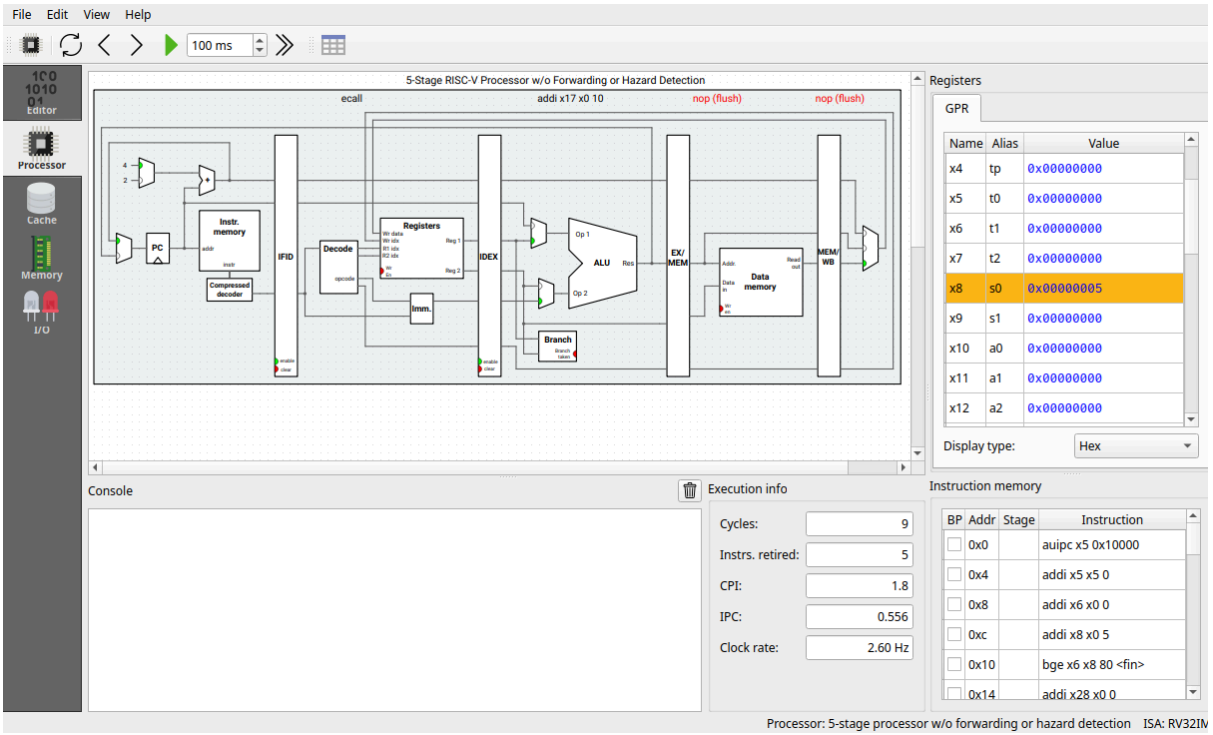
4.1 Hazards

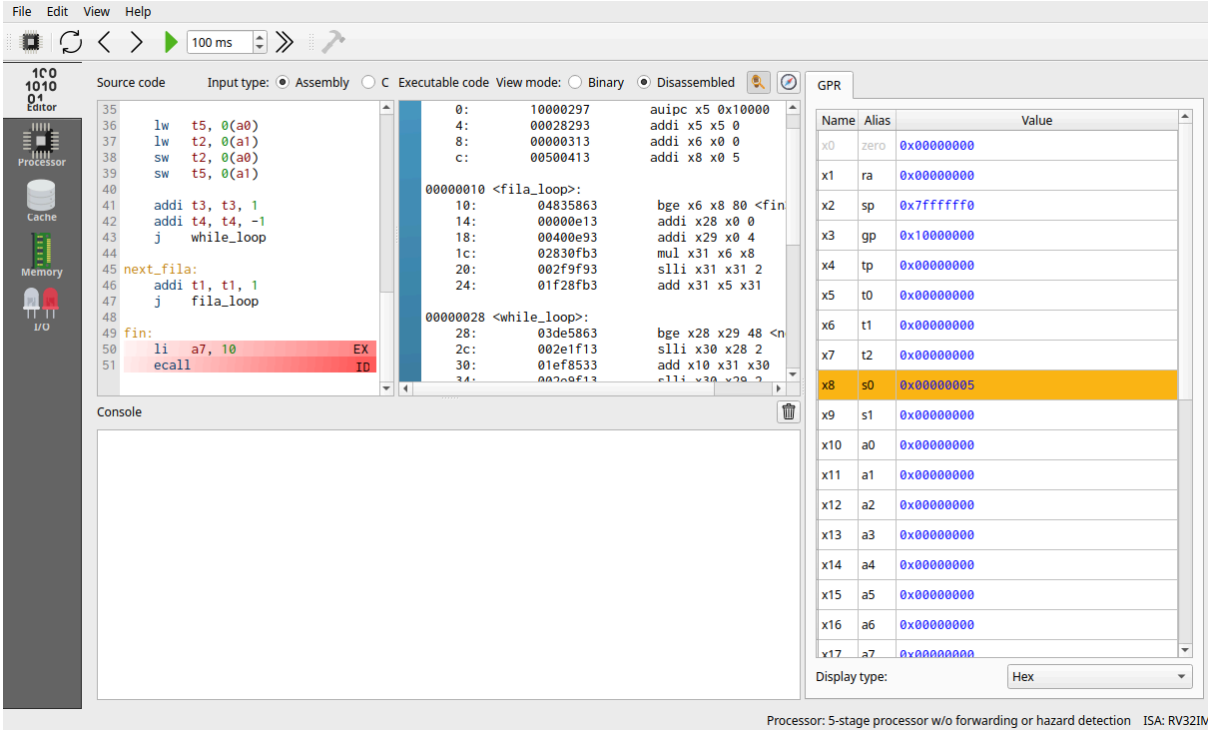
Hazard 1

Ciclo: 9

Tipo: data hazard

Registros: a7



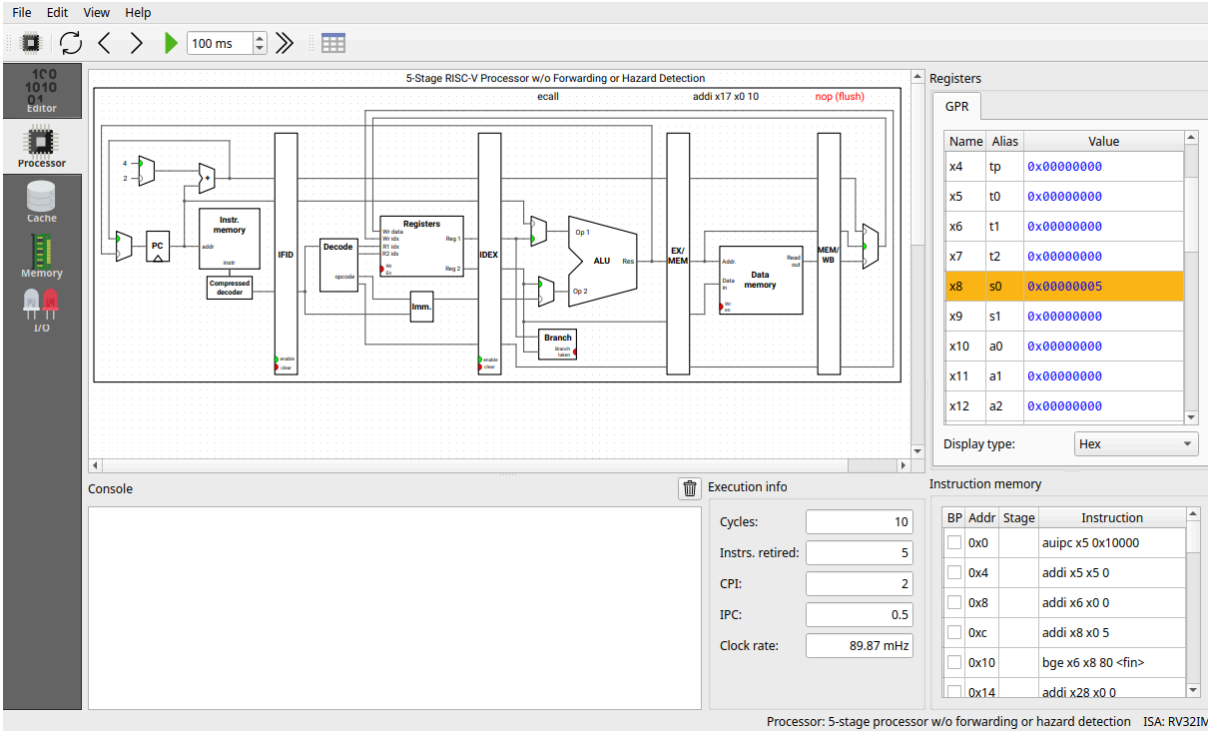


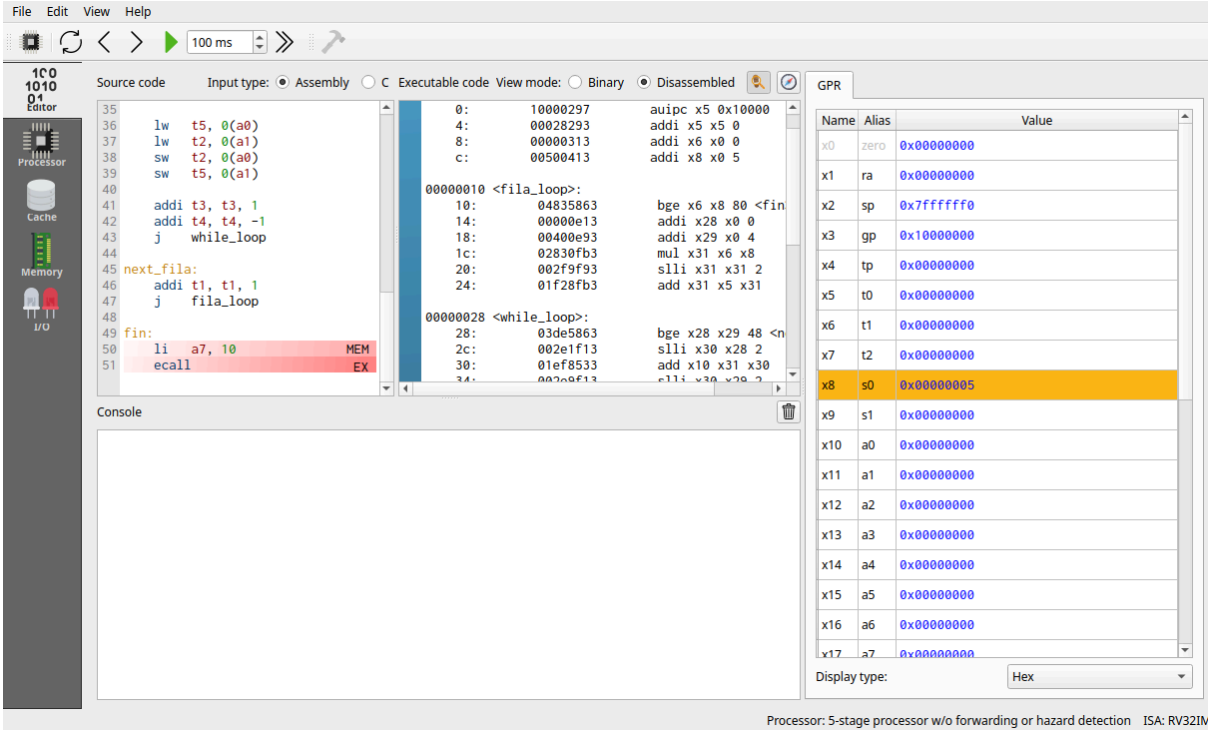
Hazard 2

Ciclo: 10

Tipo: control hazard

Registros: t1



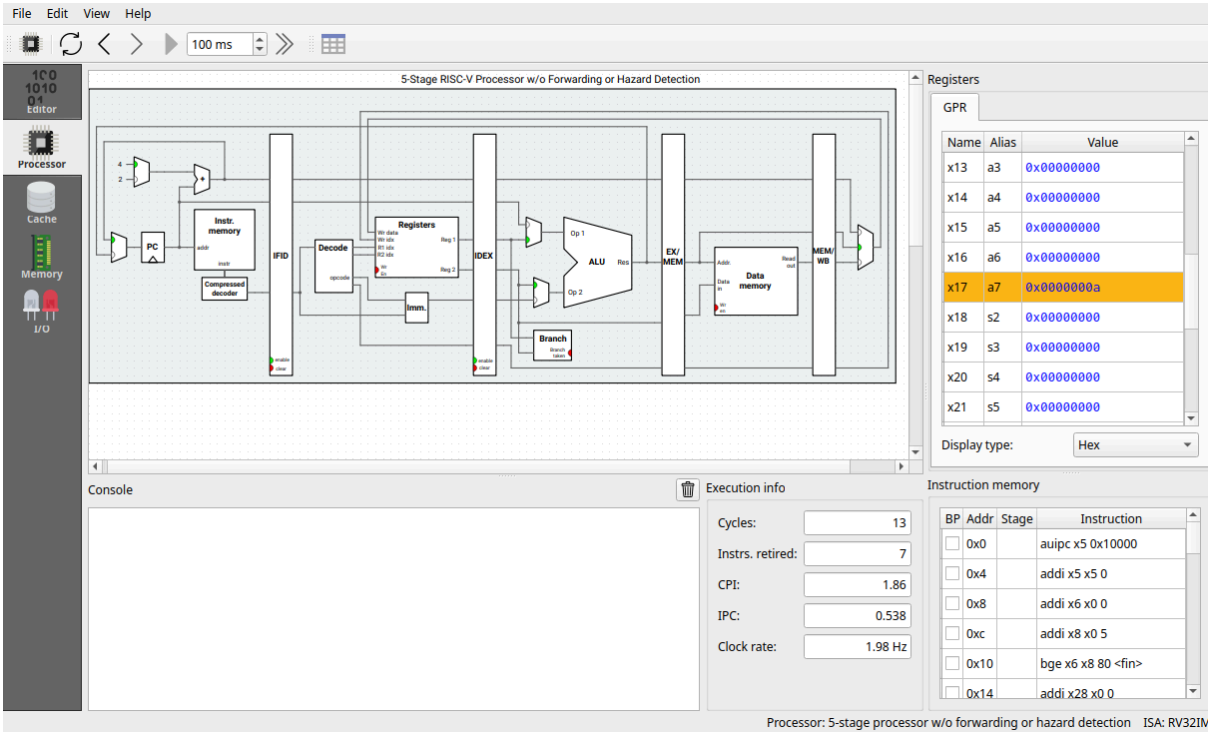


4.2 CPI

Ciclos: 13

Instrucciones: 7

CPI = 1.86



Este CPI de 1.86, se considera alto, lo ideal es que sea menor a 1, pero es resultado de los branches que al utilizarse en un ciclo, aumentan las burbujas obtenidas, rompiendo el pipeline. Aunque solamente un branch aumenta el CPI, al sumarse de forma cíclica el CPI aumenta significativamente.

4.3 Análisis Caché Original

Métrica	Hits	Misses	Rate
Original	1,932,252	14120	0.7%

Hits = D refs - D1 misses = 1,932,252

```
==16975== Copyright (C) 2002-2024, and GNU GPL'd, by Nicholas Nethercote et al.
==16975== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==16975== Command: ./Imagen_Original
==16975==
--16975-- Warning: Cannot auto-detect cache config, using defaults.
--16975--          Run with -v to see.
==16975==
==16975== I refs:        3,590,800
==16975== I1  misses:        1,185
==16975== LLi misses:        1,175
==16975== I1  miss rate:      0.03%
==16975== LLi miss rate:      0.03%
==16975==
==16975== D refs:        1,946,372  (1,683,446 rd   + 262,926 wr)
==16975== D1  misses:       14,120  (    7,488 rd   +   6,632 wr)
==16975== LLd misses:       13,991  (    7,368 rd   +   6,623 wr)
==16975== D1  miss rate:       0.7% (      0.4%     +     2.5%  )
==16975== LLd miss rate:       0.7% (      0.4%     +     2.5%  )
==16975==
==16975== LL refs:          15,305  (    8,673 rd   +   6,632 wr)
==16975== LL misses:        15,166  (    8,543 rd   +   6,623 wr)
==16975== LL miss rate:        0.3% (      0.2%     +     2.5%  )
```

4.4 Análisis Caché Optimizado

Métrica	Hits	Misses	Rate
Optimizado	2,231,779	14122	0.6%

```
==17091== Copyright (C) 2002-2024, and GNU GPL'd, by Nicholas Nethercote et al.
==17091== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==17091== Command: ./Imagen_Optimizado
==17091==
--17091-- Warning: Cannot auto-detect cache config, using defaults.
--17091--          Run with -v to see.
==17091==
==17091== I refs:        4,077,540
==17091== I1  misses:        1,187
==17091== LLi misses:        1,177
==17091== I1  miss rate:      0.03%
==17091== LLi miss rate:      0.03%
==17091==
==17091== D refs:        2,245,901  (1,958,015 rd   + 287,886 wr)
==17091== D1  misses:       14,122  (    7,489 rd   +   6,633 wr)
==17091== LLd misses:       13,991  (    7,368 rd   +   6,623 wr)
==17091== D1  miss rate:       0.6% (      0.4%     +     2.3%  )
==17091== LLd miss rate:       0.6% (      0.4%     +     2.3%  )
==17091==
==17091== LL refs:          15,309  (    8,676 rd   +   6,633 wr)
==17091== LL misses:        15,168  (    8,545 rd   +   6,623 wr)
==17091== LL miss rate:        0.2% (      0.1%     +     2.3%  )
```

4.5 Cálculo AMAT

AMAT = Hit_Time + (Miss_Rate × Miss_Penalty)

Hit_Time = 1 ns | Miss_Penalty = 100 ns

Original

Miss_Rate: 0.7%

AMAT=1+(0.007×100)=1+0.7=1.7 ns

Optimizado

Miss_Rate: 0.6%

AMAT=1+(0.006×100)=1+0.6=1.6 ns

Mejora porcentual

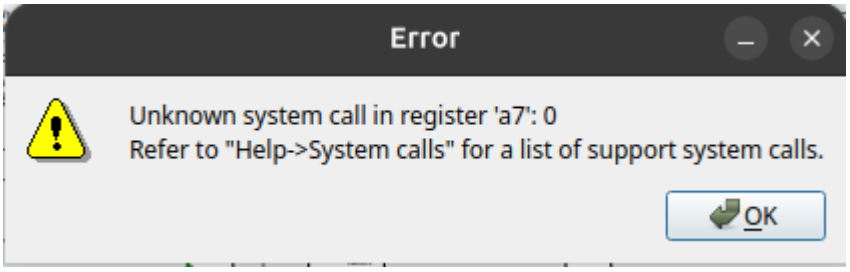
$$\frac{Antes - Despues}{Antes} * 100$$

Esta fórmula da como resultado una mejora porcentual de 5.882%

5. Análisis y Discusión

5.1 Análisis de Hazards

Los hazards son los momentos donde la siguiente instrucción no se da por alguna razón, el dato necesario no está listo, o la predicción de la siguiente instrucción es incorrecta por algún condicional como bge. En nuestro caso, tuvimos dos hazards, el primero tuvo lugar cuando el dato a7 no tiene la información en el momento en el que el programa lo esta pidiendo, hay una captura sobre ese error, que aparece explícitamente en RIPES:



El segundo Hazard que se registró, fue porque el procesador predice el camino que va a tomar el programa, cuando hay un condicional, en nuestro caso bge t1, s0, fin, este condicional crea un espacio (nop) porque el procesador asume que va a seguir la instrucción que sigue de él, pero al cumplirse va a fin, por lo tanto, el procesador crea un espacio que es donde se cancela la predicción.

5.2 Impacto de Caché

El análisis de la Caché se realizó con cachegrind. En una matriz de 320 x 313, que son más de 300 KB y se compararon dos versiones, la versión Original y la versión Optimizada.

La versión original cuenta con un algoritmo de inversión que intercambia elemento a elemento, lo que genera grandes saltos entre elementos.

La versión Optimizada cuenta con un algoritmo de inversión por agrupación, es decir, se agrupan cierta cantidad de elementos, se invierten y aprovechando la posición en que están, los mueve a la posición deseada.

Según el AMAT obtenido los tiempos son 1.7ns y 1.6ns respectivamente, por lo que el tiempo por acceso del algoritmo optimizado, es 0.1ns inferior al algoritmo original. Dando como resultado una mejora porcentual de un 5.882%, aunque parezca poca, se puede ver como un cambio básico como una agrupación tuvo efectos positivos. La optimización logra disminuir la distancia de los saltos del original.

6. Conclusiones

6.1 Hallazgos Principales

En el algoritmo utilizado en RIPES, que es el mismo algoritmo original, con una matriz de 5x5, podemos ver que se registran dos NOPs o burbujas, el primero el forwarding lo soluciona obteniendo el valor antes de que esté en un estado de memoria como tal, ya que el primero necesita un dato no registrado al momento de ser necesitado y el segundo es porque la predicción del procesador es incorrecta, cuando se hace el salto a 'fin'. Esto muestra cómo se rompe la pipeline que sigue el procesador para ejecutar las instrucciones.

El CPI mayor a 1 indica que la pipeline no logra llegar a su máximo rendimiento, por los problemas mencionados anteriormente, las burbujas NOPs. Además, se sabe que cuando el branch (bge) cambia la dirección que predice el procesador, se obtiene un ciclo de reloj adicional.

El algoritmo optimizado logra una mejora de 5.882%, con casi la misma cantidad de misses que el algoritmo original, esto sugiere que el problema recae sobre la caché, donde tenemos que hay 32 KB frente a una matriz de elementos de más de 300 KB.

Por lo tanto, se puede observar que la pequeña mejoría del algoritmo optimizado, 5.882%, no es únicamente culpa del acceso a memoria, también es culpa de la pipeline, ya que en el caso del bge, toma o sigue un camino equivocado costando un ciclo de reloj.

6.2 Aprendizajes de Arquitectura

La necesidad de herramientas, como cachegrind, para observar la tasa de fallos de caché, en este algoritmo que intercambia elemento por elemento contra el algoritmo de agrupación.

La utilización de branches, como bge, lleva a una mala CPI, por el camino que sigue el procesador prediciendo movimientos, logrando que no se llegue a un mejor rendimiento.

Cuando la cantidad de elementos es demasiado grande y la comunicación se realiza con la RAM, el tiempo de ejecución es aún mayor a si se manejan volúmenes más pequeños. Una solución a la poca mejora porcentual sería una mucho más agresiva que maneje mucha más complejidad algorítmica, como la transposición de matrices. Se puede ver en el código optimizado realizado, que logra una mejora pequeña, pero la complejidad aumentó.

El uso de las herramientas como RARS, RIPES o cachegrind, es una forma efectiva de ver la mejora de forma numérica y realizar una optimización de software que logre optimizar, elementos como el CPI y los misses de caché.